The Noble Build Notes

Last Modified Wednesday July 29 2026

OpenFOAM Tools Configuration Introduction

In this walkthrough, OpenFOAM is built as the penultimate tool on top of a large pile of supporting tools. Many are aware of one another, and conventions are therefore organized accordingly around OpenFOAM.

OpenFOAM uses a custom build tool called `Allwmake`. Similar to other `make` tools, `Allwmake` only builds files in subfolders containing an `Allwmake` file. You can run `./Allwmake` multiple times, and it just picks up wherever it left off.

Although built from a relatively simple pair of `Allwmake` commands, OpenFOAM uses a complex and, for me, somewhat opaque configuration and build process under the hood that automatically sets and edits multiple environment variables and creates numerous new paths and folders. As part of the normal documented build process, users are directed to source the `OpenFOAM/etc/bashrc` in the root OpenFOAM directory to kick off this process. This `bashrc` file generates or changes 60 environment variables (a reverse-engineered list can be found here[1])

Some of the expected configuration functionality was broken for my Ubuntu instance out of the box, in particular regarding external tool mappings. OpenFOAM uses a `etc/config.sh/tool-name` convention for settings paths, along with an associated, user-created `etc/prefs.sh` file for customization and overriding. However, in addition to defining variables in these `tool-name` files, there is also some embedded logic in some cases, and that logic didn't always end up producing the expected results, in my case.

Instead, I wound up needing to both add a `prefs.sh` file and also overwrite some of the values and/or logic set in some of those `etc/config.sh/tool-name` files as well. In particular, I needed to both include a `prefs.sh` file, and override the `metis`, `kahip`, `scotch`, and `zoltan` files before I could build their associated artifacts (`libmetis.so`, `libkahip.so`, etc) without issues. Even with all those changes, I still had to call the `prefs.sh` file ¡i¿again¡/i¿, after sourcing `bashrc`. The entire tools configuration infrastructure is mostly a black box to me, so I can't really explain why this was the case. I've included all the file edits here in the `OpenFOAM` folder inside this repository.

By convention, all the tools installed for this build use a lower-case naming convention and installed to the `/opt` folder. The OpenFOAM build is installed to `/opt/openfoam`.

Stale ParaView Libraries

I built the current version of ParaView after trying to build `v5.12.1`, the version used in the tarball associated with OpenFOAM `v2606` (the version used in this document) on OpenFOAM.com's website. Brittle dependencies exist between VTK and QT 5 and OpenFOAM's only ParaView-aware component, a module named PVFoamReader. PVFoamReader is also the only tool that requires HDF5 support. There is also another associated visualization component, an in-process off-screen renderer, that uses VTK libraries as well.

Due to the issues with those incompatible ParaView libraries, I rename the `Allwmake` files to `xxxAllwmake` in the following `openfoam` locations:

- `src/plugins/bindings/vtk-hdf`
- `src/modules/visualization`

Ignored Third-Party Tools

- ADIOS2
- CCMIO
- HDF5
- MGridGen

[1] `Notes/OpenFOAM-Environment-Variables.md`

Tool Installation Order

Core GPU/MPI Libraries

- Nvidia Driver
- CUDA Toolkit
- UCX
- libevent
- hwloc
- OpenPMIx
- PRRTE
- OpenMPI

OpenFOAM Tools

- ParaView
- Umpire
- Hypre
- Apt Installs (CGAL, Boost, etc)
- FFTW
- KaHIP
- GKlib
- METIS
- ParMETIS
- Scotch
- Zoltan
- AMGX
- PETSc
- OpenFOAM
- AmgX4Foam

Ubuntu 24.04.4 Installation

Choose a minimal install, and do not install any Nvidia drivers yet.

1) Inside your Windows instance, download the Ubuntu 24.04.4 iso file

```
Go to https://releases.ubuntu.com/24.04.4/
```

2) To boot from a usb stick, follow these directions:

```
https://ubuntu.com/desktop/docs/en/latest/how-to/create-a-bootable-usb-stick/
```

From your BIOS, boot from the USB drive and install to the target drive.
¡i¿"Install latest Graphics and Wifi hardware drivers"¡/i¿ was left ¡b¿unchecked¡/b¿ for this document.
You may require a safe boot reconfiguration[2] during the driver-less install.

Nvidia drivers

Register the CUDA Keyring and Install Drivers

```
# register the CUDA keyring
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64
    /cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
```

[2] Notes/Driverless-Install.md

```
sudo apt update

# Haswell needs Compute Architecture sm_61
sudo apt install nvidia-driver-570

# Threadripper needs Compute Architecture sm_86 and sm_120
sudo apt install nvidia-driver-580-open
reboot
```

Probe the device with `nvidia-smi`:

```
# On Haswell
nvidia-smi
| NVIDIA-SMI 570.211.01             Driver Version: 570.211.01     CUDA Version
    : 12.8

# On TR
nvidia-smi
NVIDIA-SMI 580.173.02   Driver Version: 580.173.02  CUDA Version: 13.0
0  NVIDIA GeForce RTX 3090 Ti
1  NVIDIA GeForce RTX 5090

nvidia-smi --query-gpu=gpu_name,compute_cap --format=csv
name, compute_cap
NVIDIA GeForce RTX 3090 Ti, 8.6
NVIDIA GeForce RTX 5090, 12.0
```

CUDA Toolkit

```
# Choose v12.8 to maintain compatibility between TR's sm_120 and Haswell's
    sm_61
sudo apt install cuda-toolkit-12-8
reboot
```

`CUDA_HOME` Variable

Inside `~/.bashrc`, add the `CUDA_HOME` variable which some tools look for.

```
...
export CUDA_HOME=/usr/local/cuda
export PATH=$CUDA_HOME/bin
export LD_LIBRARY_PATH=$CUDA_HOME/lib64
```

Track `PATH`, and `LD_LIBRARY_PATH` Variables

Moving forward, inside `~/.bashrc`, track changes following each successful installation of a tool:

```
...
export PATH=/opt/mytool/bin:$PATH
export LD_LIBRARY_PATH=/opt/mytool/lib:$LD_LIBRARY_PATH
```

UCX

Prerequisites

```
mkdir repos && cd repos

# collect the codebase
git clone --recursive https://github.com/openucx/ucx
cd ucx
sudo apt update
sudo apt install build-essential automake autoconf pkg-config libtool
```

Configure UCX

```
./autogen.sh

# Haswell
./configure --prefix=/opt/ucx --with-cuda=/usr/local/cuda --with-nvcc-gencode
    ="-gencode=arch=compute_61,code=sm_61" --enable-mt

# Threadripper
./configure --prefix=/opt/ucx --with-cuda=/usr/local/cuda --with-nvcc-gencode
    ="-gencode=arch=compute_86,code=sm_86 -gencode=arch=compute_120,code=sm_120
    " --enable-mt
```

OpenMPI Dependencies

Libevent

```
cd libevent
./autogen.sh
./configure --help
mkdir build && cd build
../configure --prefix=/opt/libevent
make
make verify
sudo make install
```

Hwloc

```
cd hwloc
git checkout tags/hwloc-2.14.0
./autogen.sh
./configure --help
mkdir build && cd build
../configure --with-cuda=/usr/local/cuda --prefix=/opt/hwloc
make
sudo make install
```

Prerequisites

```
sudo apt install perl python3 python3-pip virtualenv flex bison gfortran
```

OpenPMIx

```
cd openpmix
git checkout tags/v6.1.1rc2
./autogen.pl
./configure --help
mkdir build && cd build
../configure --prefix=/opt/pmix --with-hwloc=/opt/hwloc --with-libevent=/opt/
    libevent
make -j$(nproc)
sudo make install
```

PRRTE

```
cd prrte
./autogen.pl
mkdir build && cd build
../configure --prefix=/opt/prrte --with-libevent=/opt/libevent --with-hwloc=/
    opt/hwloc --with-pmix=/opt/pmix
make -j$(nproc)
sudo make install
```

OpenMPI

```
cd ompi/
./autogen.pl
mkdir build && cd build
../configure --prefix=/opt/ompi --with-libevent=/opt/libevent --with-hwloc=/opt
    /hwloc --with-pmix=/opt/pmix --with-prrte=/opt/prrte --with-ucx=/opt/ucx --
    with-ucx-libdir=/opt/ucx/lib --with-cuda=/usr/local/cuda --with-cuda-libdir
    =/usr/local/cuda/lib64/stubs
make -j$(nproc)
sudo make install
```

`ompi/etc/openmpi-mca-params.conf` Changes

Failing to do this generates an error during PETSc's `make check`. It prevents `ob1` from being the highest prioritized provider in the stack during these checks, and instead `ucx` is always picked.

```
# Point-to-point Messaging Layer
pml=ucx

# One-sided Communication Layer
osc=ucx
```

OpenMPI `openfoam/etc/prefs.sh` Changes

```
export MPI_ARCH_PATH=/opt/ompi
```

ParaView

Pre-Requisites

```
sudo apt install cmake ninja-build libtbb-dev mesa-common-dev mesa-utils
    freeglut3-dev xsltproc libxkbcommon-dev qt6-5compat-dev qt6-base-dev qt6-
    tools-dev qt6-svg-dev
```

Cmake Configuration

```
cd repos
git clone --recursive https://github.com/Kitware/ParaView.git
cd ParaView
mkdir paraview-build && cd paraview-build
cmake -GNinja -DCMAKE_INSTALL_PREFIX=/opt/paraview -DPARAVIEW_USE_PYTHON=ON -
    DPARAVIEW_USE_CUDA=ON -DPARAVIEW_USE_MPI=ON -DCMAKE_CUDA_ARCHITECTURES="61"
     -DVTK_SMP_IMPLEMENTATION_TYPE=TBB -DCMAKE_BUILD_TYPE=Release ..
```

Ninja Memory Issues

`ninja -j` runs out of memory and always had to be re-run a couple of times on both machines. Build feedback can be minimal and some libraries take several minutes to build. Somewhere around halfway, I lowered the CPU count down to around on fourth of the available cores sometimes. Apparently, this gives ninja or cmake more memory for some of the larger objects.

`ninja -j$(nproc)` completely fails on TR. Apparently, there are issues with how `ninja` handles large core sizes on some AMD CPUs. I occasionally used `cmake --build . -j$(nproc)` to complete the process.

```
ninja -j$(nproc)
# or use cmake --build . -j$(nproc)
sudo ninja install
```

ParaView and VTK `openfoam/etc/prefs.sh` Changes

```
export ParaView_VERSION="none"
export VTK_VERSION="none"
```

Umpire

Like PRRTE and OpenPMIx, Umpire is used by Hypre, and is a configurable installation inside Hypre's build process. Here is the authoritative source.

Collect Umpire Code

```
git clone --recursive https://github.com/LLNL/Umpire.git
cd Umpire
mkdir build && cd build
```

Cmake Configuration and Build

```
# Haswell
cmake -DENABLE_MPI=ON -DENABLE_CUDA=ON -DBLT_ENABLE_CUDA=ON -DBLT_ENABLE_OPENMP
    =ON -DBLT_ENABLE_MPI=ON -DBUILD_SHARED_LIBS=ON -DUMPIRE_ENABLE_CUDA=ON -
    DUMPIRE_ENABLE_OPENMP=ON -DUMPIRE_ENABLE_MPI=ON -
    DUMPIRE_ENABLE_IPC_SHARED_MEMORY=ON -DUMPIRE_ENABLE_MPI3_SHARED_MEMORY=ON -
    DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES="61"  -
    DUMPIRE_ENABLE_C=ON -DCMAKE_INSTALL_PREFIX=/opt/umpire ..

# TR
cmake -DENABLE_MPI=ON -DENABLE_CUDA=ON -DBLT_ENABLE_CUDA=ON -DBLT_ENABLE_OPENMP
    =ON -DBLT_ENABLE_MPI=ON -DBUILD_SHARED_LIBS=ON -DUMPIRE_ENABLE_CUDA=ON -
    DUMPIRE_ENABLE_OPENMP=ON -DUMPIRE_ENABLE_MPI=ON -
    DUMPIRE_ENABLE_IPC_SHARED_MEMORY=ON -DUMPIRE_ENABLE_MPI3_SHARED_MEMORY=ON -
    DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES="86;120"  -
    DUMPIRE_ENABLE_C=ON -DCMAKE_INSTALL_PREFIX=/opt/umpire ..

make -j$(nproc)
sudo make install
```

LD_LIBRARY_PATH and PATH changes in ~/.bashrc:

```
export PATH=/opt/paraview/bin:/opt/umpire/bin:/opt/ompi/bin:/opt/prrte/bin:/opt
    /pmix/bin:/opt/ucx/bin:$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=/opt/paraview/lib:/opt/umpire/lib:/opt/ompi/lib:/opt/
    prrte/lib:/opt/pmix/lib:/opt/ucx/lib:/opt/ucx/lib/ucx:$CUDA_HOME/lib64
```

Umpire etc/prefs.sh Changes

```
export UMPIRE_ARCH_PATH=/opt/umpire
```

Hypre

```
cd repos
git clone --recursive https://github.com/hypre-space/hypre.git
cd hypre
mkdir build && cd build

# Haswell
cmake -DBUILD_SHARED_LIBS=ON -DCMAKE_POSITION_INDEPENDENT_CODE=ON -
    DCMAKE_BUILD_TYPE=Release -DCMAKE_C_COMPILER=mpicc -DCMAKE_CXX_COMPILER=
    mpicxx -DHYPRE_ENABLE_CUDA=ON -DMPI_INCLUDE_PATH=/opt/ompi/include -
    DCMAKE_CUDA_ARCHITECTURES="61"  -DHYPRE_ENABLE_GPU_AWARE_MPI=ON -
    DHYPRE_ENABLE_OPENMP=ON -DHYPRE_ENABLE_UMPIRE=ON -DHYPRE_ENABLE_UMPIRE_HOST
    =ON -DHYPRE_ENABLE_UMPIRE_DEVICE=ON -DCMAKE_INSTALL_PREFIX=/opt/hypre -
    DTPL_UMPIRE_INCLUDE_DIRS=/opt/umpire/include -DTPL_UMPIRE_LIBRARIES=/opt/
    umpire/lib/libumpire.so ../src

# Threadripper
cmake -DBUILD_SHARED_LIBS=ON -DCMAKE_POSITION_INDEPENDENT_CODE=ON -
    DCMAKE_BUILD_TYPE=Release -DCMAKE_C_COMPILER=mpicc -DCMAKE_CXX_COMPILER=
    mpicxx -DHYPRE_ENABLE_CUDA=ON -DMPI_INCLUDE_PATH=/opt/ompi/include -
```

```
    DCMAKE_CUDA_ARCHITECTURES="86;120"  -DHYPRE_ENABLE_GPU_AWARE_MPI=ON -
    DHYPRE_ENABLE_OPENMP=ON -DHYPRE_ENABLE_UMPIRE=ON -DHYPRE_ENABLE_UMPIRE_HOST
    =ON -DHYPRE_ENABLE_UMPIRE_DEVICE=ON -DCMAKE_INSTALL_PREFIX=/opt/hypre -
    DTPL_UMPIRE_INCLUDE_DIRS=/opt/umpire/include -DTPL_UMPIRE_LIBRARIES=/opt/
    umpire/lib/libumpire.so ../src

make -j$(nproc)
sudo make install
```

Hypre `etc/prefs.sh` Changes

```
export HYPRE_ARCH_PATH=/opt/hypre
```

CGAL

```
sudo apt update
sudo apt install libcgal-dev libgmp-dev libmpfr-dev libboost-system-dev
```

CGAL `prefs.sh` Changes

```
#can use default cgal
export CGAL_ARCH_PATH=/usr
```

FFTW

```
# Use lscpu to identify supported architectures to enable
lscpu

cd fftw-3.3.11
mkdir build && cd build

# Haswell
../configure --prefix=/opt/fftw --enable-openmp --enable-shared --enable-
    threads --enable-mpi --enable-sse2 --enable-avx --enable-avx2

# Threadripper
../configure --prefix=/opt/fftw --enable-openmp --enable-shared --enable-
    threads --enable-mpi --enable-sse2 --enable-avx --enable-avx2 --enable-
    avx512

make -j$(nproc)
sudo make install
```

KaHIP

```
mkdir build && cd build
```

```
cmake -DCMAKE_INSTALL_PREFIX=/opt/kahip -DCMAKE_BUILD_TYPE=Release -
    DBUILD_SHARED_LIBS=ON -DMPI_HOME=/opt/ompi -DCMAKE_INSTALL_LIBDIR=/opt/
    kahip/lib ..

make -j$(nproc)
sudo make install
```

Karypis Labs

Create a [consolidated repository] containing the three Karypis Labs tools used by PETSc and OpenFOAM.

GKlib

```
cd GKlib
mkdir build && cd build

cmake -DCMAKE_INSTALL_PREFIX=/opt/karypis -DCMAKE_BUILD_TYPE=Release -DSHARED=
    ON -DOPENMP=ON ..

make -j$(nproc)
sudo make install
```

METIS

For me, METIS had a couple of bugs in the root folder's `CMakeLists.txt`. I changed these lines:

```
(line 55) include_directories(include)
...
(line 73) add_subdirectory("include")
```

Pre-Cmake Step

Uncomment the `#defines` for `IDXTYPEWIDTH` and `REALTYPEWIDTH` inside `/opt/karypis/include/metis.h`:

```
(line 33) #define IDXTYPEWIDTH 32
...
(line 43) #define REALTYPEWIDTH 64 /* note the 64 for double precision */
```

Cmake Configuration and Build

```
cd METIS
mkdir build && cd build

cmake -DCMAKE_BUILD_TYPE=Release -DSHARED=ON -DOPENMP=ON -DCMAKE_INSTALL_PREFIX
    =/opt/karypis -DGKLIB_PATH=/opt/karypis ..

make -j$(nproc)
sudo make install
```

ParMETIS

```
cd ParMETIS
mkdir build && cd build

cmake -DCMAKE_BUILD_TYPE=Release -DSHARED=ON -DOPENMP=ON -DCMAKE_INSTALL_PREFIX
    =/opt/karypis -DGKLIB_PATH=/opt/karypis -DMETIS_PATH=/opt/karypis -
    DCMAKE_C_COMPILER=mpicc ..

make -j$(nproc)
sudo make install
```

Scotch

Clone Repository

```
# Need Scotch for orthogonal decomposition
git clone --recursive https://gitlab.inria.fr/scotch/scotch.git
```

Cmake Configuration

```
cd Scotch
mkdir build && cd build

cmake -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=ON -DCMAKE_INSTALL_PREFIX
    =/opt/scotch -DINSTALL_METIS_HEADERS=OFF  -DMPI_THREAD_MULTIPLE=ON -
    DCMAKE_C_COMPILER=mpicc ..

make -j$(nproc)
sudo make install
```

Scotch `openfoam/etc/config.sh/scotch` and `openfoam/etc/prefs.sh` Changes

```
export SCOTCH_ARCH_PATH=/opt/scotch
```

Zoltan

```
cd Zoltan
mkdir build && cd build

# Use CFLAGS and CXXFLAGS to force shared libraries
../configure --prefix=/opt/zoltan --enable-mpi --with-mpi=/opt/ompi --with-
    parmetis=1 --with-parmetis-incdir=/opt/karypis/include --with-parmetis-
    libdir=/opt/karypis/lib --with-scotch=1 --with-scotch-incdir=/opt/scotch/
    include --with-scotch-libdir=/opt/scotch/lib CFLAGS="-O3 -fPIC" CXXFLAGS="-
    O3 -fPIC"

make everything -j$(nproc)

# Create the shared library version of libzoltan manually
cd src
```

```
# ran on Haswell 7/26
mpicxx -shared -o libzoltan.so *.o -L/opt/scotch/lib -lptscotch -lscotch -L/opt
    /karypis/lib -lparmetis -lmetis

# Create the system installation folders
sudo mkdir -p /opt/zoltan/lib /opt/zoltan/include

# Copy shared object library
sudo cp libzoltan.so /opt/zoltan/lib/

# Copy the Zoltan_config.h file into include
sudo cp include/Zoltan_config.h /opt/zoltan/include

# Copy the Zoltan header files into include
cd ../../src
sudo cp include/*.h /opt/zoltan/include
```

AMGX

Clone Repository

```
git clone https://github.com/amgx/amgx.git
cd amgx
mkdir build && cd build
```

Test launcher Issues

Change a line in `src/CMakeLists.txt` prior to running `cmake`:

```
(line 55) target_link_libraries(amgx_tests_launcher "/opt/ompi/lib/libmpi.so")
```

Cmake Configuration and Build

```
# Haswell
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/opt/amgx -
    DCMAKE_CUDA_ARCHITECTURES="61" -DCMAKE_C_COMPILER=mpicc -
    DCMAKE_CXX_COMPILER=mpicxx -DCMAKE_CUDA_FLAGS="-I/opt/ompi/include" -
    DCMAKE_EXE_LINKER_FLAGS="-L/opt/ompi/lib -lmpi" ..

# TR
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/opt/amgx -
    DCMAKE_CUDA_ARCHITECTURES="86;120" -DCMAKE_C_COMPILER=mpicc -
    DCMAKE_CXX_COMPILER=mpicxx -DCMAKE_CUDA_FLAGS="-I/opt/ompi/include" -
    DCMAKE_EXE_LINKER_FLAGS="-L/opt/ompi/lib -lmpi" ..


make -j$(nproc)
sudo make install
```

PETSc

Repository and Prerequisites

```
# We need PETSc for scientific computation
git clone --recursive https://github.com/petsc/PETSc.git

# We need blas and lapack for PETSc
sudo apt install libopenblas-dev liblapack-dev
```

Configure

```
cd PETSc
mkdir build

sudo mkdir /opt/petsc
sudo chown user:user /opt/petsc

# works after removing Scotch's duplicate metis.h insertion
./configure --prefix=/opt/petsc --with-openmp=1 --with-bison=1 --with-boost=1
    --with-cuda=1 --with-ucx-dir=/opt/ucx --with-mpi-dir=/opt/ompi --with-amgx-
    dir=/opt/amgx --with-hwloc-dir=/opt/hwloc --with-umpire-dir=/opt/umpire --
    with-scotch-dir=/opt/scotch --with-ptscotch-dir=/opt/scotch --with-fftw-dir
    =/opt/fftw --with-zoltan-dir=/opt/zoltan --with-hypre-dir=/opt/hypre --with
    -parmetis-dir=/opt/karypis --with-metis-dir=/opt/karypis

make PETSC_DIR=/home/user/repos/petsc PETSC_ARCH=arch-linux-c-debug all

make PETSC_DIR=/home/user/repos/petsc PETSC_ARCH=arch-linux-c-debug check

make PETSC_DIR=/home/user/repos/petsc PETSC_ARCH=arch-linux-c-debug install

sudo chown root:root /opt/petsc
```

PETSc `openfoam/etc/prefs.sh` Changes

```
export PETSC_ARCH_PATH=/opt/petsc
```

OpenFOAM

Installation Topology

Like all the external tools, this build assumes a target installation path of `/opt/openfoam`. Create a new folder in `/opt` called `openfoam`, and give temporary ownership to your regular user:

```
sudo mkdir /opt/openfoam
sudo chown username:username /opt/openfoam
cp -r repos/OpenFOAM/* /opt/openfoam
```

Copy This Repository's `OpenFOAM/etc` Contents into Target Folder

From this repostory, copy the contents of `OpenFOAM/etc` into `/opt/openfoam/etc/` and overwrite existing files.

Source `openfoam/etc/bashrc`

```
cd /opt/openfoam

# sources prefs.sh, but later bashrc overwrites some vars
source /opt/openfoam/etc/bashrc

# source prefs again, overwriting the overwrites
source /opt/openfoam/etc/prefs.sh
```

Ignore ParaView/VTK `Allwmake` Files

Rename the `Allwmake` files to `xxxAllwmake` in the following `openfoam` locations:

- `src/plugins/bindings/vtk-hdf`
- `src/modules/visualization`

Run Allwmake

```
# Execute both Allwmake and Allwmake-modules
./Allwmake -j

# Plugins
/./Allwmake-plugins -j
```

Post-Install Environment Clean-Up and Folder Consolidation

Once the installation is complete, I consolidated my environment and generated folders, and locked down the installation by granting ownership to `root`.

```
# move everything that Allwmake built into ...
# one lib folder
mv $FOAM_USER_LIBBIN/* $FOAM_LIBBIN

# and one bin folder
mv $FOAM_USER_APPBIN/* $FOAM_APPBIN

# remove dummy libs
rm -rf $FOAM_LIBBIN/dummy

# don't use a separate folder for the ompi libs
mv $FOAM_LIBBIN/sys-openmpi/* $FOAM_LIBBIN

# clean up the now empty folder
rm -rf $FOAM_LIBBIN/sys-openmpi

# move these tools into the bin folder
mv $FOAM_LIBBIN/../../tools/linux64Gcc/* $FOAM_APPBIN/

# clean up the now empty folder
rm -rf $FOAM_LIBBIN/../../tools

# just move all the lib files into a folder named lib
mv $FOAM_LIBBIN /opt/openfoam/lib

# just move all the bin files into the existing bin folder
```

```
mv $FOAM_APPBIN/* /opt/openfoam/bin

# remove the now empty legacy paths
rm -rf /opt/openfoam/platforms

exit
```

~/.bashrc Clean-Up

Now, source a new version of .bashrc with the following contents:

```
# OpenFOAM environment variables
export CUDA_HOME=/usr/local/cuda
export WM_PROJECT=openfoam
export WM_PROJECT_DIR=/opt/$WM_PROJECT
export FOAM_APPBIN=$WM_PROJECT_DIR/bin
export FOAM_LIBBIN=$WM_PROJECT_DIR/lib

# Just setting these to main application folders for now
export FOAM_SITE_APPBIN=$FOAM_APPBIN
export FOAM_SITE_LIBBIN=$FOAM_LIBBIN

# Set to user's custom projects folder
export WM_PROJECT_USER_DIR=/home/user/My-OpenFOAM-Stuff
export FOAM_USER_APPBIN=$WM_PROJECT_USER_DIR/bin
export FOAM_USER_LIBBIN=$WM_PROJECT_USER_DIR/lib
export FOAM_RUN=$WM_PROJECT_USER_DIR/run
export WM_THIRD_PARTY_DIR=none

export PATH=$FOAM_USER_APPBIN:$FOAM_APPBIN:/opt/scotch/bin:/opt/karypis/bin:/
    opt/kahip/bin:/opt/fftw/bin:/opt/umpire/bin:/opt/paraview/bin:/opt/ompi/bin
    :/opt/prrte/bin:/opt/pmix/bin:/opt/hwloc/bin:/opt/libevent/bin:/opt/ucx/bin
    :$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$FOAM_USER_LIBBIN:$FOAM_LIBBIN:/opt/petsc/lib:/opt/amgx/
    lib:/opt/zoltan/lib:/opt/scotch/lib:/opt/karypis/lib:/opt/kahip/lib:/opt/
    fftw/lib:/opt/hypre/lib:/opt/umpire/lib:/opt/paraview/lib:/opt/ompi/lib:/
    opt/prrte/lib:/opt/pmix/lib:/opt/hwloc/lib:/opt/libevent/lib:/opt/ucx/lib:
    $CUDA_HOME/lib64
```

Check Paths

From a new terminal, check the paths are resolved.

```
ldd /opt/openfoam/lib/libOpenFOAM.so
ldd $FOAM_APPBIN/simpleFoam
# other tools, libs, etc...
```

Finally, lock the folder down:

```
sudo chown root:root /opt/openfoam
```

AmgX4Foam

Hack wmake/cuda

Out of the box, AmgX4Foam's default configuration options limit only a single CUDA arch code, so I changed the behavior of the -cu flag so that it is no longer responsible for passing the architecture. Instead,

I replaced line 12 (`cuARCH := -m64 -arch=sm_$(NVARCH)`) in the `wmake/cuda` file:

```
# Haswell
(line 12) cuARCH    :=  -m64 -gencode arch=compute_61,code=sm_61

# Threadripper
cuARCH    :=  -m64 -gencode arch=compute_86,code=sm_86 -gencode arch=
    compute_120,code=sm_120
```

However, I still had to enable CUDA for the compilation by including the flag though: `./Allwmake -cu 1`.

Lines 17 in `src/Make/options` and 12 in `wmake/c++` were also removed.

I also had to add 3 lines to the top of `src/Make/options`:

```
LIB_SRC = $(FOAM_SRC)
AMGX_INC = /opt/amgx/include
AMGX_LIB = /opt/amgx/lib
```

Finally, I also had to add these two lines to `EXE_INC` and `LIB_LIBS` accordingly:

```
EXE_INC = \
    ...
    -I/usr/local/cuda/include \
    -I/opt/ompi/include \
    ...

LIB_LIBS = \
    ...
    -L/usr/local/cuda/lib64 -lcudart \
    ...
```

AmgX4Foam's `src/csrMatrix/csrMatrix.h` was also missing a `#include <cuda_runtime.h>` statement.

Run `Allwmake`

```
./Allwmake -cu 1
```

By default, this installs to the `FOAM_USER_LIBBIN` folder, whether or not that folder exists. Note that `FOAM_USER_LIBBIN` typically uses OpenFOAM's variable-based naming conventions: `~/OpenFOAM/<user name>-<version>/platforms/<arch><compiler><precision><index size><third party folder>/lib`.